

Backend Performance Optimization Report

Project Overview

This project is a cloud-hosted business management application built using:

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose

The application manages:

- Parties
- Orders
- Items
- Payment summaries
- Business reports

During testing, several APIs exhibited high latency despite the database containing a relatively small amount of data.

A structured performance investigation was conducted to identify and eliminate bottlenecks.

Investigation Methodology

The optimization process followed a systematic approach:

Step 1: Measure Total API Response Time

Measured complete API execution time for all major endpoints.

Step 2: Measure Database Query Time

Instrumented Mongoose queries to determine database contribution.

Step 3: Measure Full Backend Processing Time

Measured total server-side execution.

Step 4: Measure JSON Serialization Time

Measured `JSON.stringify()` overhead.

Step 5: Measure Dataset Size

Recorded:

- Number of parties
- Number of orders
- Number of items
- Records processed

Step 6: Measure Payload Size

Measured response size sent to clients.

Step 7: Query Pattern Analysis

Reviewed:

- Query frequency
- Repeated lookups
- Aggregation pipelines
- Authentication middleware

Step 8: Database Index Audit

Analyzed:

- Existing indexes
- Missing indexes
- Query access patterns

Initial Findings

Dataset Size:

- Parties: 7
- Orders: 4
- Items: 15

Despite the small dataset:

Endpoint	Initial Response Time
Report All	1000ms - 3350ms
Report Single	650ms - 900ms

Endpoint	Initial Response Time
Orders By Party	150ms - 800ms
Payment Summary	250ms - 1500ms

This immediately indicated that dataset size was not the primary bottleneck.

Bottleneck #1: N+1 Query Pattern

Problem

The report generation endpoints repeatedly executed:

```
Order.findOne() Order.findOne() Order.findOne() Order.findOne()
```

for every processed record.

Instead of:

1 query fetching all required records

the application performed many sequential database roundtrips.

Impact

Each MongoDB Atlas request incurred network latency.

Latency accumulated across multiple sequential queries.

Solution

Replaced repeated lookups with:

- Batch queries
- Aggregation pipelines
- Preloaded datasets

Result

Report endpoints immediately dropped from:

1000ms - 3350ms

to:

150ms - 300ms

Bottleneck #2: Authentication Database Lookup

Problem

Every authenticated request executed:

```
User.findById(decodedToken._id)
```

This occurred on:

- Orders API
- Reports API
- Payment APIs
- Party APIs
- Item APIs

Result:

One additional database roundtrip for every request.

Observed latency:

30ms - 300ms per request

Solution

The JWT already contained:

```
```json
```

```
{
 "_id": "...",
 "email": "...",
 "username": "..."
}
```

Auth middleware was changed to:

```
req.user = {
 _id,
```

```
 email,
 username
 }
```

without querying MongoDB.

Database lookup is now only used as a fallback for legacy tokens.

## Result

Before:

```
GET /current-user
272ms
311ms
```

After:

```
GET /current-user
18ms
4ms
```

Improvement:

**93% - 99% reduction**

---

## Bottleneck #3: Missing Query Indexes

### Index Audit

The following indexes were introduced and verified:

#### Orders

```
orderSchema.index({
 partyId: 1,
 createdAt: -1
})
```

Optimizes:

- Party order lookup
  - Payment summary aggregation
  - Sorted order retrieval
- 

## Items

```
itemSchema.index({
 partyId: 1,
 outgoingDate: 1
})
```

Optimizes:

- Report generation
  - Date-range filtering
- 

```
itemSchema.index({
 orderId: 1
})
```

Optimizes:

- Order joins
  - Bulk item retrieval
- 

```
itemSchema.index({
 partyId: 1
})
```

Optimizes:

- Party-specific item queries
-

## Parties

```
partySchema.index({
 partyName: 1,
 createdBy: 1
}, { unique: true })
```

Optimizes:

- Party lookup
- Duplicate prevention

---

## Bottleneck #4: Connection Pool Configuration

### Previous State

MongoDB driver used near-default connection pool settings.

Potential issues:

- Connection queuing
- Atlas latency spikes
- Cold connection creation

### New Configuration

```
mongoose.connect(uri, {
 maxPoolSize: 20,
 minPoolSize: 5,
 socketTimeoutMS: 45000,
 serverSelectionTimeoutMS: 5000,
 heartbeatFrequencyMS: 10000,
 retryWrites: true,
 retryReads: true
})
```

Benefits:

- Reduced connection creation overhead
- Better concurrency handling
- Faster recovery from transient network issues

# JSON Serialization Analysis

Measured serialization time:

0.01ms  
0.15ms  
0.80ms  
1.40ms  
4.10ms (worst)

Conclusion:

JSON serialization contributed less than 1% of total response time.

No optimization required.

---

## Payload Size Analysis

Measured payload sizes:

Endpoint	Payload
Payment Summary	0.15 KB
Report Single	4.44 KB
Orders By Party	7.12 KB

Conclusion:

Network transfer size was not a bottleneck.

---

## Final Performance Results

### Report All

Before:

1000ms - 3350ms

After:

74ms - 131ms

Best-case improvement:

**95.8%**

Worst-case improvement:

**92.6%**

---

## Report Single

Before:

650ms - 900ms

After:

144ms - 190ms

Improvement:

**70% - 84%**

---

## Orders By Party

Before:

150ms - 800ms

After:

104ms - 175ms

Improvement:

**Up to 87%**

---

## Payment Summary

Before:

250ms - 1500ms

After:

74ms - 121ms

Improvement:

**Up to 95%**

---

## Engineering Lessons Learned

1. Measure before optimizing.
  2. Small datasets can still produce slow APIs.
  3. N+1 queries can be more expensive than large datasets.
  4. Authentication middleware can become a hidden bottleneck.
  5. Proper indexing should be based on actual query patterns.
  6. MongoDB Atlas latency becomes significant when many sequential queries are executed.
  7. Connection pooling impacts scalability and consistency.
  8. Instrumentation is essential for identifying true bottlenecks.
- 

## Final Outcome

The backend was transformed from a system with multi-second response times into a consistently responsive application.

Key achievements:

- ✓ Eliminated N+1 query patterns
- ✓ Removed authentication database lookup from every request

- ✓ Added query-driven MongoDB indexes
- ✓ Optimized aggregation pipelines
- ✓ Improved MongoDB connection pooling
- ✓ Reduced report generation latency by over 90%
- ✓ Reduced common API response times to approximately 70ms - 190ms

The resulting architecture is significantly more scalable, efficient, and production-ready.